

Table 3. Partially Explicit Intermediate Representations and Partially Explicit Decision Making

Classification		Papers
Representation learning/graph learning		[40, 74, 137, 148, 153, 210]
Natural language processing		[7, 13, 39, 41, 43, 44, 93, 96, 117–119, 141, 152, 157, 188, 189, 204?, 205]
Computer vision		[1, 6, 10, 12, 18, 45, 58, 72, 73, 92, 106, 113, 127, 155, 164, 171, 193, 195, 203]
Reinforcement learning		[5, 11, 22, 80, 82, 88, 97, 98, 101, 111, 121, 132, 134, 138, 162, 165, 167, 172, 199]
Mathematics / symbolic regression		[54, 59, 60, 83, 107, 125, 143, 147, 177]
Logic and knowledge processing	Concept/rule learning	[20, 25, 169, 185]
	Logical reasoning	[29, 32, 55, 56, 66, 84, 115, 126, 139, 142, 173, 181, 196, 206]
Applications	Visual question answering	[128, 168, 187, 201]
	Medical	[70, 86, 99]
	Communication	[175, 176]
	Programming	[15, 57, 94, 174]
	Recommended system	[37, 122]
	Security	[179, 192]
	Others	[19, 53, 100]

mathematical expression. Hence, the process is partially explicit and interpretable. The logical decision-making part of DSR can also be divided into two steps: the explicit evaluation process and the implicit adjustment process. The former refers to completing the calculation of the fit or reward of the expression based on the comparison of the expression and the data. The latter is intended to provide feedback on the degree or reward to the RNN to guide the subsequent expression generation process, which relies on RNN gradient descent or other optimization algorithms and is achieved by adjusting the weight of RNN, so it is also a partially explicit process.

In Part I, we mentioned the NSPS(neuro-symbolic program search) method proposed by [172] to improve the automation level of autonomous driving system design by automatically searching and synthesizing neural symbolic programs. This method uses neural networks to extract rich information required for autonomous driving, such as vehicle speed, acceleration, and attitude, from the input data. However, it cannot be directly used for symbolic logic processing. Therefore, NSPS bridges the two by defining a DSL(domain-specific language), which contains basic primitives for driving attributes, such as vehicle speed and acceleration, and statements for implementing advanced priors. However, the method automatically selects and combines these operations through program search and is, therefore, partially explicit. However, the operations defined by DSL can directly reflect the logic of driving decisions, so they are also partially explicit.

For the same reason, the final driving decision of the decision-making process is obtained by automatically searching and combining a given set of neural-symbolic operations. This process relies on the implicit expression of the neural network weights and activation functions. Although the symbolic operations provide a certain degree of interpretability, specific working methods of the underlying neural network model and program search algorithm are still implicit, which leads to the overall decision-making logic of NSPS being partially explicit.

Finzel et al. [74] proposed using GNN(graph neural networks) to classify relational data and verify the GNN output by generating understandable explanations combined with ILP(inductive logic programming). This method first uses

GNN to extract features from graph-structured data. These data include patterns in the Kandinsky pattern data set, which are composed of different geometric objects such as circles and triangles, as well as attributes such as shape, color, size, and location of the node. GNN aggregates the information of neighbor nodes through the connection relationships between them, updates the feature vectors of nodes, and identifies the graph structure that has the most significant impact on the classification results through interpreters such as GNN-Explainer. These graph structures and their importance scores are then converted into facts and rules in a Prolog program. In this case, symbolic logic exists in the form of ILP, which uses these transformed symbolic data as background knowledge to learn rules that describe the logic of classification decisions. These rules are not only based on structural features such as spatial relationships between nodes and node attributes such as color and shape but also comprehensively consider the importance scores of these features to generate understandable, logical rules that explain GNN classification decisions. The intermediate representation of this method converts the output of GNN into Prolog facts and rules suitable for ILP processing and builds a bridge from neural network features to symbolic logic. Therefore, this intermediate representation combines the two forms of latent vector embedding and Prolog rules and is partially explicit. Similarly, the decision-making logic in this method involves both two parts. The feature representation automatically learned by the neural network from the data is used as the input of the ILP, and it makes subsequent decisions based on this representation. Therefore, the two are partially explainable.

3.3 Explicit Intermediate Representations or Explicit Decision Making(prediction)

There are 3 neuro-symbolic AI studies in this category, as Table 4 shows, and all of which have three characteristics. First, neural networks are used to extract features from data. However, the representations of these features cannot be directly processed by symbolic logic, so they must use intermediate representations to fill the gap. Second, either intermediate representations or overall decision logic is entirely explicit.

Table 4. Explicit Intermediate Representations OR Explicit Decision Making

Classification	Papers
Entity linking	[102]
NVM based robotic manipulation	[105]
Question and Answering	[103]

Jiang et al. [102] proposed an entity link prediction method based on LNN, LNN-EL(Logical Neural Network-Entity linking) to solve the entity linking problem in short texts, detailed in the multimodal non-heterogeneous Neuro-Symbolic AI section in Part I of this article. This method converts a set of logical rules into the network structure of LNN in LNN. It accurately matches the mentioned entities by processing features extracted from text or knowledge graph data. As mentioned in Part I 3.4, LNN maps logical operations directly into neural networks. Its neural network and symbolic logic operate the same data type in the same representation space without an intermediate representation to bridge the two. At the same time, the activation state of each neuron or neuron group can directly correspond to the truth value state of a logical proposition, so the decision-making process of logical operations is more accessible to explain. However, for LNN-EL, using deep models in the feature extraction stage results in the intermediate representation between its output and the LNN input being only partially explicit. Therefore, even if the LNN network structure is used in the inference stage, it is only moderately interpretable.

Kapanipathi et al. [103] proposed a NSQA(neural symbolic question answering) system based on semantic parsing and reasoning. The method first uses an AMR(Abstract Meaning Representation) to convert natural language questions into AMR graphs according to explicit linguistic rules. Then it uses a neural network model to identify and link entities and relationships, that is, entities mentioned in the text through entity linking match entities in the knowledge base, and use relationship links to match actions or relationships in the AMR graph with attributes or relationships in the knowledge base to convert the AMR graph into an intermediate representation corresponding to the entities and relationships. This representation is then further converted into logical queries, and a Logical Neural Network reasoner is used to infer based on the execution of these queries. The introduction of neural networks in the semantic parsing stage and entity and relationship linking stage of NSQA brings a certain degree of interpretability impact to the method. Although the output of these models can be directly verified by looking at the predictions on specific inputs, we still rate them as moderately explainable, considering that they are not as transparent as rule-based systems.

Katz et al. [105] proposed integrating high-level reasoning and low-level action control in robot manipulation tasks. This method is based on the neural virtual machine structure and can simulate the execution of a Turing-complete, purely symbolic virtual machine. The method first defines the NVM's (neural virtual machine) configuration, including layer size and connections between layers. It determines the source code of the symbolic algorithm to be executed on the neural virtual machine. Then, based on the initial configuration, an NVM instance with appropriate initial weights is automatically generated. A unique activity pattern vector represents each symbol in the source code. The compilation process of the source code is to apply a series of rules, such as fast storage-erasure learning, to the connection weight update of NVM, thereby encoding the symbolic algorithm into NVM. The compiled program can be executed by running the recursive dynamics of NVM. At each time step, the activity patterns of different layers in NVM represent the changes in the virtual machine state, and the weight matrix represents the program logic and data storage. Finally, the method uses a bidirectional lookup table called a "codec" to perform encoding and injecting symbolic input or extracting and decoding symbolic output at any time. This approach also means that the user can check the simulated machine state or provide input in symbolic form. It is worth mentioning that the decision-making logic of this method is implemented through changes in neural activation patterns, so the neural network's output uses the exact representation as the processing object of the logical symbol. However, although the two do not require an intermediate representation to bridge the two, their representation space is the high-dimensional vector space of the neural network. Hence, the overall decision logic needs to be more transparent and fully interpretable.

3.4 Explicit Intermediate Representations and Explicit Decision Making(prediction)

There is 1 neuro-symbolic AI study in this category, which has three characteristics. The most significant difference between this classification and the previous one is that the intermediate representation and the overall decision logic are explicit. However, an intermediate representation is still needed to fill the gap between extracted features and symbolic processing.

Kimura et al. [108] et al. (2021) proposed a neural symbolic framework to solve reinforcement learning problems in text-based games. This method first extracts basic propositional logic from text observations obtained in the environment through a semantic parser, converting natural language text into a logical expression form. Then, external knowledge bases such as ConceptNet are used to understand the semantic categories of words in the text and refine the extracted propositional logic. Finally, the extracted propositional logic and the lexical category information obtained from ConceptNet are combined through the FOL(First Order Logic) converter and converted into first-order logical facts that reflect the specific characteristics and conditions of the game state. These logical facts are subsequently used as